

## RESEARCH ARTICLE

# A Domain-Agnostic Agentic Architecture for Structured Extraction of Engineering Knowledge

Mehdi Tarkian<sup>a</sup>, Oscar Ikechukwu<sup>a</sup>, Sanjay Nambiar<sup>a</sup> and Marie Jonsson<sup>a</sup>

<sup>a</sup>Division of Product Realization, Department of Management and Engineering, Linköping University, Linköping, Sweden.

### ARTICLE HISTORY

Compiled February 23, 2026

### ABSTRACT

Engineering organizations increasingly adopt large language models (LLMs) to retrieve facts from heterogeneous technical documentation; however, deployments remain brittle when answers must be grounded to specific sections, pages, and tabular data in documentation. In practice, retrieval-augmented generation and agent pipelines often fail due to hallucinations, weak validation, and limited traceability, especially in table-dense and revision-controlled engineering corpora. This paper presents an applied, domain-agnostic agentic architecture for structured extraction and verification of engineering knowledge. The approach introduces a persistent, SQL-backed Relational Control Plane (RCP) that stores goals, strategies, tool calls, outputs, and validation outcomes as first-class relational entities. The design is to integrate and supports governance by storing every intermediate artifact and check result. Reasoning is organized as a six-stage loop (goal definition, strategy selection, agent execution, agent validation, strategy validation, and goal validation) that enforces evidence-centric progression and enables deterministic replay and audit-ready provenance. We evaluate the framework on two industrial case studies involving a hydraulic product catalogue and revision-controlled technical documentation. Compared to baseline retrieval and non-validated agent pipelines, the proposed architecture improves citation and revision fidelity, reduces unsupported generations, and provides transparent, queryable execution traces. These results demonstrate a practical pathway for deploying trustworthy generative-AI systems in engineering knowledge management.

### KEYWORDS

Agentic AI; Engineering Informatics; Large Language Models; Knowledge Engineering

## 1. Introduction

Recent advances in large language models (LLMs) have intensified efforts to capture and operationalize institutional knowledge at scale. Engineering organizations, in particular, must retrieve precise details about products, components, and processes by cross-referencing specifications, standards, service manuals, engineering change orders (ECO/ECN), and part-revision histories—tasks that traditionally require years of tacit know-how. Retrieval-Augmented Generation (RAG) improves access to such knowledge, yet RAG pipelines hallucinate, struggle with multi-document reasoning, and underperform on non-text modalities common in engineering (dense tables, figures, CAD callouts, scanned PDFs) [1–3].

Advanced RAG. Methods such as RAPTOR build hierarchical representations of long documents and retrieve at multiple abstraction levels, improving cross-chunk coherence and reducing irrelevant context [4]. While RAPTOR helps with concept-level queries (e.g., “torque requirements across assemblies”), engineering settings still face persistent hurdles: (i) revision and change awareness (answers must reflect the correct revision/serial range and supersession rules); (ii) table/units fidelity (facts often live in tables and require robust parsing and normalization); (iii) cross-constraint reasoning across BOMs, ECOs, service bulletins, and standards; and (iv) traceability to exact clauses, figures, or table cells for audits [1–4].

Why tables and PDFs remain hard. Even with better retrieval, accuracy hinges on reliably extracting structure and numbers from PDFs. Large-scale studies show table detection and structure recognition remain challenging; dependable parsing and unit normalization often require specialized datasets and tools (e.g., PubTables-1M, GROBID/GROBID-quantities) that standard LLM pipelines rarely embed by default [5–7].

Agent frameworks mitigate some limitations by orchestrating tools, multi-step planning, and verification (e.g., ReAct-style reasoning-and-acting; tool-use learning) [8,9]. In engineering, however, no single agent suffices: depending on the domain and need, teams compose domain-specific agents—a table/units agent for numeric extraction and normalization, a revision/lineage agent for change control, a CAD/PLM agent for configuration, and a standards agent for normative rules—dramatically increasing the burden of tracking state, node/edge flows, retries, error routes, and observability. Frameworks such as LangGraph help, but productionizing typically still requires formal state design, graph wiring, heterogeneous tool integration (vector stores, OCR, table parsers, CAD/PLM APIs), persistence, evaluation, and replay [10–12].

This study presents a generic agentic framework that builds upon LangGraph by introducing a domain-agnostic control loop to standardize orchestration, validation, and state management across diverse application domains. The framework structures agentic reasoning into six coordinated stages—goal definition, strategy selection, agent execution, agent validation, strategy validation, and goal validation—ensuring that every reasoning step is both purposeful and verifiable.

A key innovation lies in its relational persistence model, where all orchestration artifacts—including goals, strategies, agent calls, parameters, outputs, and validation outcomes—are stored in an SQL-backed control plane rather than transient in-memory states. This approach unifies planning and execution while providing a coherent, queryable representation of the system’s behavior. The persistent state enables full observability, deterministic replay, and governance through audit-ready records. By externalizing orchestration policies and maintaining independence from specific retrieval or vector-store technologies, the framework achieves high modularity, traceability, and

adaptability, significantly reducing setup complexity and operational overhead.

Empirical evaluations on engineering document retrieval and reasoning tasks demonstrate that this architecture retains the benefits of multi-step retrieval and tool use while improving reliability through structured validation. Beyond accuracy, the central applied contribution is operational: the system produces audit-ready traces that support governance and reproducibility in industrial settings where incorrect units, revision scope, or missing provenance can cause costly downstream errors.

The main contributions of this work are:

- (C1) a validation-centric, six-stage agentic control loop for evidence-grounded extraction from heterogeneous engineering documents;
- (C2) an SQL-backed Relational Control Plane (RCP) that persists goals, strategies, function calls, artifacts, and validation outcomes for inspection and deterministic replay;
- (C3) an applied evaluation on two industrial case studies, including baseline comparisons and metrics for citation quality, unit/revision fidelity, and latency/cost.

### ***1.1. Agentic AI in Manufacturing***

Agentic AI systems are characterized by autonomous decision-making, goal-oriented action, and coordination among multiple agents, often operating with limited supervision in dynamic environments [13,14]. Unlike traditional AI rule-based or single-model classification—agentic AI uses reinforcement learning, multi-agent systems (MAS), and LLMs to continuously reason, plan, and act in response to changing conditions [15].

LLMs are used to extract design knowledge from engineering documents. Nicholson et al. (2025) evaluate GPT-4V (a vision-enabled LLM) on extracting information from 2D mechanical drawings. In one experiment, GPT-4V could describe a part and correctly read off five key dimensions from a drawing [16]. It then generated CAD scripts (in CadQuery, FeatureScript, OpenSCAD) to model the part, iteratively refining errors. This shows that VLMs can partially automate drawing interpretation and CAD generation, though the authors note that GPT-4V’s limited short-term memory required corrective feedback in multiple prompts [16]. In document knowledge management, Freire et al. (2024) build an LLM-based Q&A system that ingests a factory’s documentation and operator knowledge. Their system answers operator queries by searching manuals and past logs. A user study found that the LLM tool dramatically sped up information retrieval and problem resolution, though workers still preferred human mentors when available [17]. These results emphasize the promise of LLMs for mining engineering knowledge, as well as the ongoing need for careful validation of the answers they provide.

Another challenge is hallucination and factual grounding. VLMs like GPT-4V and GPT-4 are prone to “hallucinating” plausible-sounding but incorrect details [18]. Systems try to mitigate this by using external knowledge sources: for example, Gkournelos et al. embed the LLM in a digital twin and knowledge base. They explicitly give the agent access to “manufacturing proprietary knowledge bases” and live data via a digital twin, so it can verify its outputs [18]. A related issue is the limited context and memory of LLM agents. Most systems use one or two conversational agents with fixed context windows. Nicholson et al. found that even GPT-4V loses track of earlier details when prompted iteratively through many CAD corrections [16]. Singh et al. note that continuous processes (batch vs continuous) may require “more reprompts” and that

LLMs’ finite context will limit how much of the system model can be processed at once [19]. They suggest RAG as a solution to feed the LLM only the necessary structured slices of the digital twin [19]. In practice, this means splitting knowledge into a database or memory store and having LLM agents fetch relevant facts as needed.

There is a challenge in integrating agentic AI with formal validation frameworks or redundant checks to catch erroneous agent outputs before they cause defects. The literature shows a rapid rise of multi-agent, LLM-driven automation for engineering. These systems demonstrate the potential of agentic AI to improve flexibility and autonomy. Yet they also reveal gaps: existing methods often scale only to narrow tasks, rely on fixed architectures, and incur hallucinations without grounding. Future research must tackle these gaps by designing truly dynamic agent systems, embedding structured knowledge (databases, digital twins) for LLMs, and building rigorous validation layers. As multiple researchers observe, combining LLM reasoning with dedicated knowledge resources and modular agent coordination is essential for safe, scalable manufacturing automation

## 2. Materials and Methods

This work introduces a generic agentic framework that extends the capabilities of LangGraph by abstracting orchestration into a domain-agnostic control loop. The proposed architecture standardizes agent coordination, validation, and state management, thereby substantially reducing the cost and complexity of setup across different application domains. The framework operationalizes agentic reasoning through a structured six-stage process: goal definition, strategy selection, agent execution, agent validation, strategy validation, and goal validation. The controller executes a six-stage loop:

Normalize user query into a Goal object with explicit success criteria: target variable(s), required evidence granularity (e.g., table cell citation), applicable revision scope, and acceptable uncertainty.

Choose a strategy template (e.g., Revision-Aware Spec Lookup, Table-Grounded Numeric Extraction, Cross-Document Conflict Resolution). A strategy defines what kinds of agents may be invoked and in which order e.g. search, parse, normalize and verify.

Dynamically select one or more agents whose preconditions match the current state (e.g., product ID known vs. unknown) and whose outputs advance the strategy’s targets (e.g., “obtain torque value + units + source cell”).

Run local checkers for schema completeness, unit normalization, range plausibility, document/section anchors, and failure diagnostics. Failed checks trigger retries or alternate agents.

Test whether the collected artifacts meet the plan’s sufficiency conditions, e.g., at least two corroborating sources, consistent revisions, no unresolved conflicts. If not, the controller loops back to step 3 within the same strategy or escalates to a broader strategy.

Apply the goal acceptance predicate to confirm correct revision and serial range, exact citations (cell/figure/paragraph), and uncertainty within thresholds. Only then is the answer finalized.

This loop decouples progress monitoring (validation gates) from capability invocation (agents), which makes the system robust to partial failures and amenable to targeted improvements.

A distinguishing characteristic of this framework is its relational persistence model.

Instead of maintaining transient, in-memory states typical of conventional agent frameworks, all orchestration artifacts—goals, strategies, agent invocations, parameters, outputs, validations, and errors—are persisted in an SQL database. This design yields a coherent, queryable state representation that unifies planning and execution. The SQL-backed control plane simplifies observability through dashboards and lineage queries, supports systematic evaluation through deterministic replays, and facilitates governance through audit-ready traces. By decoupling orchestration from any specific vector store, retriever, or external tool provider, the system remains modular and adaptable to heterogeneous data environments.

### ***2.1. Evaluation setup, baselines, and metrics***

To match industrial deployment requirements and the Applied Artificial Intelligence emphasis on comparative evaluation, we assess the proposed framework against two baselines: (i) a standard retrieval-augmented generation (RAG) pipeline (retrieve top- $k$  chunks  $\rightarrow$  generate answer), and (ii) an agentic pipeline with tool use but without the RCP persistence model and validation gates.

Across both case studies, we report: (a) answer correctness against an annotated query set; (b) citation accuracy (whether cited passages/table cells contain the claimed fact); (c) unit fidelity (correct unit and numeric value, including conversions); (d) revision/applicability fidelity (answer is valid for the requested revision/serial range); and (e) operational cost (end-to-end latency and token/compute budget). We operationalize hallucination rate as the fraction of responses containing at least one unsupported factual claim (i.e., claim not found in the retrieved evidence).

### ***2.2. Implementation and resources***

The system is implemented as a modular pipeline consisting of: (i) a document harvesting stage (PDF parsing/OCR, table reconstruction, and normalization into an SQL “harvested” database); (ii) an orchestration stage (LangGraph-based controller with the SQL-backed RCP for execution traces); and (iii) an application layer for interactive querying. Implementation code for the two case studies is available at [https://github.com/oscik559/Hydroscand\\_Produktbok](https://github.com/oscik559/Hydroscand_Produktbok) and [https://github.com/oscik559/Project\\_Saab](https://github.com/oscik559/Project_Saab). To support reproducibility and deployment planning, we report (where permitted): the size of the document corpus (pages/tables), the number of evaluation queries, approximate person-months for implementation and integration, and the compute footprint per query (LLM calls, tokens, and runtime).

### ***2.3. Storing Engineering Data***

Engineering documents interleave structured tables, free text, illustrative figures, reference notes, revision markers and domain-specific notations. To accommodate this heterogeneity, the framework adopts a two-tier, SQL-based storage model: a persistent knowledge base that objectifies core engineering data, and a transient in-session cache for run-time data assembly. Together, these stores decouple stable long-term knowledge representation from dynamic reasoning throughout the control loop.

The Harvested database serves as the canonical repository of engineering knowledge extracted from source documents. Regardless of domain, raw material—PDFs, scanned documents, manuals, or CAD-derived metadata—are converted into a normalized re-

lational form. This process, referred here as objectification, ensures that: (a) semantic structure and entities are represented consistently across documents, (b) numerical and unit-bearing fields are stored with canonical types, (c) provenance (page, section, figure, or table anchors) is retained, and (d) the data becomes amenable to deterministic, query-based retrieval. This structured store represents the domain knowledge base available to all strategies and agent execution. It remains stable across sessions and can be extended incrementally when new documents or revisions are introduced. Since it captures the extracted content directly, without reasoning artefacts, it provides the foundation for reproducibility and auditability within the overall architecture.

## Figure Placeholder

**Figure 1.** Sample of distillation of engineering knowledge in a technical documentation

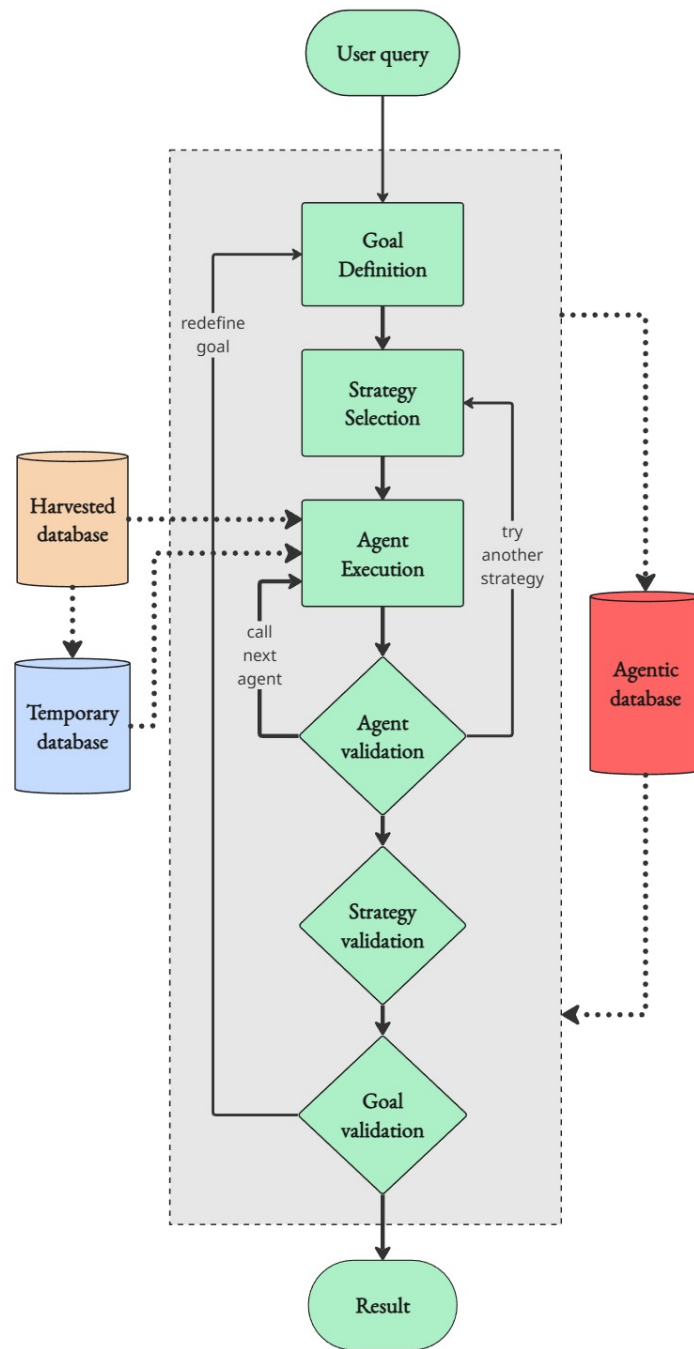
During reasoning, strategy plans often require intermediate aggregation, filtering, or transformation of data that should not modify the persistent knowledge base. For this purpose, the framework constructs a temporary, session-scoped database at the start of each control-loop instance. This store is used to hold: (a) intermediate subsets produced by search or filtering agents, (b) normalised or reformatted values required for downstream validation, (c) assembled evidence packets combining multiple knowledge sources, and (d) transient artefacts generated during multi-step strategies. The temporary database is strictly short-lived: its contents exist only for the duration of the reasoning episode and are fully captured through the RCP as run-time artefacts. This design ensures that every intermediate transformation remains traceable while preventing contamination of the canonical extracted data.

### ***2.4. Relational Control Plane***

The Relational Control Plane (RCP) serves as the central orchestration database of the proposed framework, externalizing coordination logic from code into a normalized relational schema. It provides a generic and auditable foundation for agentic reasoning, particularly within engineering domains where precision, traceability, and validation are essential. Conceptually, the RCP enforces a clear separation between design-time policy and run-time execution. At design time, the library tables define the system’s capability space—strategies are represented as reusable plan templates, while agent functions are modeled as atomic operations with declared input–output contracts. At run time, corresponding instance tables record each concrete goal, strategy, and function invocation, including their parameters, outputs, validation outcomes, and diagnostic messages. This design allows behavior to be authored once and reused across multiple domains, while also transforming orchestration into persistent, versioned data that can be Analysed and refined without modifying the underlying control graph.

Upon receiving a query, the controller instantiates a goal record with explicit success criteria—such as revision scope, evidence granularity, or citation requirements—and selects an appropriate strategy instance according to stored policies in the RCP libraries.

The controller then dynamically dispatches one or more agents whose input–output



**Figure 2.** Enter Caption

contracts match the current system state. Typed contracts are enforced directly within the RCP: each function template defines its expected schema and post-conditions, while validation outcomes at the function, strategy, and goal levels are persisted as structured records in the instance tables. Progress through the loop is therefore validation-centric, with advancement permitted only when defined predicates—such as unit normalization, revision applicability, or citation exactness—are satisfied. All computational artifacts (including structured tables, retrieved passages, intermediate results, and evidence links) are stored within the RCP, whereas the orchestration graph itself maintains only lightweight references, such as the active goal, strategy, or function identifiers. This arrangement minimizes in-memory state, enables reproducible runs, and provides complete observability through standard relational queries.

Compared to conventional agentic frameworks—including basic LangGraph deployments in which logic and execution order are often hard-coded—the RCP introduces three fundamental advances.

- (1) Changes in execution order, eligibility, or capability preference (for example, inserting a lineage-checking step before finalization or prioritizing hierarchical retrieval over keyword search) are achieved through updates to policy data rather than through code rewrites.
- (2) Every decision, artifact, and validation outcome is recorded, enabling deterministic replay, post hoc analysis, and quantitative evaluation of strategy performance.
- (3) New capabilities or alternative implementations (e.g., retrievers, table parsers, or unit normalization tools) can be added by simply registering them within the RCP library, allowing existing strategies to incorporate them immediately without altering the orchestration graph.

Collectively, these properties deliver provenance and audibility by design, while significantly lowering the cost of adapting the framework across domains and document types.

The novelty of the proposed framework lies in its treatment of orchestration as data. Strategies, agent contracts, and validation rules are elevated to persistent, queryable entities within the RCP, while the graph controller functions primarily as an interpreter of these declarative policies. The RCP enables data-driven policy updates rather than architectural redesigns. Thus, providing a foundation of generality, audibility, and adaptability that current agentic frameworks lack.

## 2.5. *Strategy Library*

The Strategy Library defines a set of reusable, templated plans that encapsulate standardized patterns of tool usage, reasoning, and validation across different engineering contexts. Each strategy represents a distinct problem-solving blueprint that specifies four core components: targets, eligible agents, sufficiency conditions, and fallback policies.

The targets describe the evidence or variables required for successful completion of the goal, such as a numerical value, physical unit, revision identifier, or document citation. Eligible agents define the permissible computational modules and their preferred execution order, for example, a workflow that begins with search, proceeds to table parsing and unit normalization, and concludes with revision verification. The sufficiency conditions express the level of coverage and consistency necessary to consider a strategy complete, such as requiring two corroborating sources or one source explicitly documenting revision supersession. Finally, fallback policies specify adaptive

## Figure Placeholder

**Figure 3.** strategy library

behaviors when a strategy fails to meet its sufficiency threshold, such as expanding the search from a specific product to a broader component class or triggering hierarchical retrieval when local searches provide inadequate coverage.

### 2.6. *Agent Library*

The Agent/Function Library contains modular, atomic tools that execute the functional operations defined within strategies. Each agent is designed as a self-contained, side-effect-free component with declared inputs, outputs, and post-conditions. Agents can be dynamically composed to fulfill the operational steps of a strategy without modification to the orchestration graph, ensuring both modularity and maintainability.

## Figure Placeholder

**Figure 4.** Agent library

Typical agents include components for text retrieval, hierarchical retrieval, table extraction, unit normalization, revision and lineage checking, figure and callout linking, keyword expansion, table filtering, and lightweight evidence scoring. The text retrieval agent performs keyword- and semantic-based searches within document repositories, supporting constraints such as product identifiers, revision levels, or document classes. Hierarchical retrieval agents extend this functionality by constructing and traversing document trees to surface content at varying structural depths, thereby improving conceptual recall in complex documents. Table extraction and structuring agents identify tabular regions within PDFs, reconstruct headers and merged cells, and emit normalized data with explicit cell anchors. Unit normalization agents detect and convert physical quantities into canonical units while maintaining source provenance. Revision and lineage checking agents trace supersession chains, serial-range applicability, and effective dates to ensure that data conforms to the correct revision state. Additional agents manage figure or callout linkage, keyword and entity expansion, and filtering of candidate tables or rows using rule-based or learned relevance functions. Lightweight evidence scoring agents assess the adequacy of the gathered evidence relative to the strategy’s targets, guiding the system toward sufficiency or escalation when necessary.

Together, these agents form the operational substrate of the framework. Each produces validated artifacts that feed directly into a strategy’s target set—comprising elements such as value, unit, revision, and citation. The strategy’s validation gates then determine whether these artifacts collectively satisfy the defined sufficiency conditions. If the conditions are unmet, the orchestration layer automatically invokes additional

or alternative agents. Because all agents adhere to uniform interface specifications, individual components can be replaced or extended—such as substituting a standard retriever with a hierarchical variant—without any modification to the orchestration logic. This design principle ensures interoperability, robustness, and adaptability in multi-agent reasoning across diverse engineering document domains.

### 2.7. *In-Session Instantiation*

The Relational Control Plane (RCP) provides the persistent substrate that binds design-time definitions to run-time execution. At design time, the Strategy Library (Section 3.2) and the Agent Library (Section 3.3) declare the capability space in terms of reusable strategy templates and atomic agent functions, each with typed input–output contracts and post-conditions. At run time, these templates are instantiated as in-session records that capture the concrete evolution of a reasoning episode. For every user query, the controller creates a goal instance with explicit acceptance criteria and then materializes a strategy instance selected from the Strategy Library. Within this strategy context, the controller dynamically instantiates one or more agent functions from the Agent Library. Each instantiation records its parametrization, inputs consumed, outputs produced, and the results of function-, strategy-, and goal-level validations.

## Figure Placeholder

**Figure 5.** Overview of the in-session instantiation process

The in-session tables thus serve as a normalized execution ledger. Inputs and outputs are stored as first-class artifacts linked to their originating strategy and agent instances, together with citations, revision lineage, unit normalizations, and diagnostic messages. The orchestration graph maintains only stable identifiers (e.g., current goal, strategy, and function IDs), while all heavy artifacts—retrieved passages, structured tables with cell anchors, intermediate calculations, and validation reports—reside in the RCP. This arrangement enables deterministic replay and post hoc inspection, supports fine-grained provenance from final answers back to specific clauses or table cells, and provides a coherent basis for evaluation and governance. Because the controller reads policies from the libraries and writes outcomes to the in-session ledger, the behavior can be adapted by updating library entries rather than rewriting orchestration logic. New strategies or agents become immediately available for instantiation once registered, and their inputs/outputs are automatically captured under the same schema. In effect, the RCP operationalizes a data-driven “goal  $\rightarrow$  strategy  $\rightarrow$  agent  $\rightarrow$  validation” loop: strategies and functions defined in the libraries are realized as concrete, auditable instances, and all inputs, outputs, and validations are persistently recorded, ensuring consistent state management and traceable multi-agent reasoning across domains.

### 3. Results

#### 3.1. Case Study I:

##### *Product Catalogue Documentation (Hydroscand).*

Industrial hydraulic systems rely on extensive product catalogues that consolidate specifications for hoses, fittings, and coupling assemblies. These multi-page documents encode critical attributes such as dimensions, pressure ratings, reinforcement structures, material compositions, thread standards etc. The Hydraulic Produktbok [20], a widely used Nordic fluid-power reference from Hydroscand AB, a leading provider of hydraulic and industrial hose solutions, is representative of this class: it presents chapter-organised product families, construction descriptions, and dense specification tables spanning numerous variants within each product line.

The catalogue’s structure poses notable challenges for automated knowledge extraction. Variant-level data appear in compact tables with merged cells, multi-row headers, mixed unit conventions, and visually implied hierarchies. Thread standards (BSP/G, JIC, ORFS, NPTF) depend on subtle typographic distinctions that OCR and naive PDF parsers often distort. Free-text descriptions (entirely in Swedish) use specialised terminology that impede generic retrieval models. Consequently, conventional approaches such as keyword search or chunk-based RAG tend to lose table semantics, misinterpret values, or fail to deliver cell-level, engineering-grade citations. General-purpose retrievers also lack the domain grounding needed for queries based on technical specifications, such as pressure-rating lookup, thread-compatibility checks or application-based product selection.

This environment thus presents an appropriate proof-of-concept domain for the proposed framework. It combines heterogeneous modalities (tables, free text, mixed typography), domain-specific naming conventions, and a need for strict validation of engineering attributes. Successfully demonstrating reliable extraction and reasoning here directly tests the framework’s ability to mitigate hallucination, preserve unit fidelity, and deliver verifiable outputs with explicit provenance.

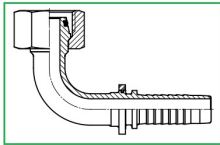
To operationalize extraction and reasoning, the framework is instantiated through a layered architecture that separates document processing, agent orchestration, and application-level interaction.

##### *3.1.1. Layer 1: Document Processing/Extraction*

The extraction layer reconstructs the foundational domain knowledge-base upon which the agentic reasoning layer operates into a normalized relational database (harvested.db).

Each data page is pre-processed at hi-res rendering to preserve typographic and geometric fidelity. A crucial step here, since subtle distinctions—notation spacing (e.g., “G 1/2” vs. “G1/2”), sub/superscripts, merged header cells, or unit annotations—carry semantic meaning for standards and dimensional specifications.

Parsed content are inserted into a relational schema: *categories* (e.g., Högtrycksslang, Presskopplingar), *families* (e.g., Kappaflex 2K, T9090) and *product-level variants* (e.g., 1105-10-04, 1059-00-16). The objective here is not merely to extract product data, but to reconstruct the catalogue’s implicit structure: the hierarchical organisation of categories and product families, the fine-grained geometry of specification tables, and the contextual metadata that defines material properties, thread standards, and operating limits.

| KAPITEL 4:2 PRESSKOPPLINGAR                                                       |            |                                                                                                                                    |          |          |
|-----------------------------------------------------------------------------------|------------|------------------------------------------------------------------------------------------------------------------------------------|----------|----------|
|  |            | <div>4274~2</div> <div>M-gängad</div> <div>90° böj, inv.</div> <div>24° tätningsskiva med O-ring</div> <div>PRODUKTGRUPP 300</div> |          |          |
| Artikelnr.                                                                        | Gänga      | Rör                                                                                                                                | Typ      | Slang ID |
| 4274-06-03-2                                                                      | M 14 x 1,5 | 6 mm                                                                                                                               | DKSO 90° | 3/16"    |
| 4274-06-04                                                                        | M 14 x 1,5 | 6 mm                                                                                                                               | DKSO 90° | 1/4"     |
| 4274-08-03-2                                                                      | M 16 x 1,5 | 8 mm                                                                                                                               | DKSO 90° | 3/16"    |
| 4274-08-04-2                                                                      | M 16 x 1,5 | 8 mm                                                                                                                               | DKSO 90° | 1/4"     |
| 4274-10-04-2                                                                      | M 18 x 1,5 | 10 mm                                                                                                                              | DKSO 90° | 1/4"     |

**Figure 6.** Document preview during extraction (Case 1)

By decoupling extraction from reasoning, the system ensures that all downstream tasks operate on validated, structured data rather than raw PDF content.

### 3.1.2. Layer 2: Agentic Reasoning

The agentic reasoning layer operationalises the six-stage control loop described in Section 2.4. A user query (e.g., specification lookup, compatibility assessment, or family-level aggregation) is normalised into a structured *goal instance* with explicit evidence requirements (target attribute, value type, and validation predicate). The controller then selects an appropriate *strategy instance* and executes an ordered *function plan*. All orchestration artifacts—including goal, strategy selection, function invocations, inputs, outputs, and validation outcomes—are persisted in the relational control plane (*agentic.db*), enabling replay, inspection, and failure attribution at record level.

While Section 2.4–2.7 define the generic control loop and RCP schema, this section focuses on their concrete instantiation for catalogue-based reasoning. All described artifacts correspond to rows in the relational control plane and are not conceptual abstractions.

#### Strategy Templates:

Product catalogue queries exhibit a small number of recurring patterns. In this case study, most requests fall into four archetypes: (i) direct specification lookup for a known product code, (ii) contextual search with attribute constraints, (iii) compatibility checks between components, and (iv) aggregation across product families. These archetypes are encoded as declarative strategy templates in the *StrategyLibrary* table, each specifying a target, an ordered plan skeleton, and a sufficiency condition that determines whether the goal is satisfied. Table 1 summarises an exemplary strategy set instantiated for the catalogue domain.

#### Agent Library:

Agent functions are tools capabilities that populate strategy plans. In this implementation, functions are exposed through a standard interface (*success: bool*, *result: dict*) and are registered in *FunctionTemplateLibrary* together with declared inputs/outputs and postconditions. Table 2 lists the core function set used in the hydraulic catalogue case.

**Table 1.** Strategy templates for hydraulic catalogue queries.

| ID | Strategy                  | StrategyDescription                                                       | PlanSteps                                                                                               | Sufficiency Condition                             |
|----|---------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| 1  | Specification Lookup      | Retrieve a known specification from an identified product code.           | Extract product code → Query database → Extract attributes → Analyse with LLM                           | Attribute found and unit normalized.              |
| 2  | Contextual Product Search | Identify candidate products matching textual or semantic constraints.     | Extract requirements → Semantic search → Filter items → Extract attributes → Analyse with LLM           | At least one valid product returned.              |
| 3  | Compatibility Check       | Assess coupling or thread compatibility between two hydraulic components. | Extract component specifications → Match standards → Check dimensional compatibility → Analyse with LLM | Standards matched; no dimensional conflict.       |
| 4  | Family Summary            | Aggregate specification data across all variants within a product family. | Retrieve family variants → Extract attributes → Aggregate results → Analyse with LLM                    | Full variant set retrieved and schema consistent. |

**Table 2.** Agent function library for the hydraulic catalogue domain.

| ID | Function Name                          | Purpose (Example Output)                                                                                                                                 |
|----|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <code>func_search_products</code>      | Retrieve candidate products using keywords and filters.<br><i>Output:</i> List of product codes with summary specifications.                             |
| 2  | <code>func_filter_by_attribute</code>  | Narrow a product list by applying attribute constraints (e.g., pressure, DN size).<br><i>Output:</i> Filtered list satisfying the specified constraints. |
| 3  | <code>func_get_product_details</code>  | Retrieve complete specifications and provenance for a given product.<br><i>Output:</i> JSON object with metadata and catalog page reference.             |
| 4  | <code>func_extract_attributes</code>   | Extract target attribute values from a product specification.<br><i>Output:</i> Attribute values with units and citations.                               |
| 5  | <code>func_normalize_units</code>      | Convert values to canonical units while preserving original formats.<br><i>Output:</i> 420 bar → 42 MPa.                                                 |
| 6  | <code>func_analyse_data</code>         | Compose the final answer with citations and verified source trace.<br><i>Output:</i> Grounded explanation linked to catalog data.                        |
| 7  | <code>func_extract_product_code</code> | Identify and validate a product identifier from a free-text query.<br><i>Output:</i> Validated product code (e.g., 4201-16-16).                          |

Notably, `func_analyse_data`<sup>1</sup> performs a structured LLM synthesis over verified intermediate artifacts. It is instructed to preserve catalogue terminology (e.g., “Arbetsstryck”, “Gängtyp”), avoid unsupported claims, and reference source-provenance fields. All other functions operate deterministically over structured records (from `harvested.db`), enabling validation gates to intercept failures before an answer is returned.

These functions adhere to a standardized interface (success: bool, result: dict), enabling orchestration via LangGraph without bespoke code. As a result, adding new capabilities (e.g., `func_check_temperature_range`) involves no orchestration modification—only policy entry registration.

### 3.1.3. Layer 3: Application-level interaction

The application layer exposes the reasoning system through a Flask-Based Web Interface. It offers an interactive front-end tailored for exploratory use and inspection. It accepts free-form queries, which are normalized and submitted to the controller as goal instances. The interface provides real-time visualizations of the reasoning process, including progress through strategy stages, intermediate agent outputs, and final structured answers. Crucially, all responses are enriched with provenance annotations: source, page numbers, and references presented alongside normalized attributes to preserve interpretability.

### 3.1.4. Worked Example:

To illustrate end-to-end behaviour, this subsection traces a representative specification query:

*“Vilken gängstorlek har en 4201-16-16?”*  
(“What thread size does a 4201-16-16 have?”)

The system must:

- (i). recognise the product code,
- (ii). retrieve the corresponding catalogue record,
- (iii). extract the requested attribute (thread size), and
- (iv). generate a grounded response.

Although the query is simple, it exercises the framework’s defining elements: explicit separation of goals, strategies, and functions, and validation gates that prevent incomplete or unsupported answers.

#### *Goal definition:*

Execution begins at `GoalDefine`, where the free-text query is normalised into a structured goal instance with explicit evidence requirements. An LLM extracts key entities (product: “4201-16-16”; attribute: “gängstorlek”) and the controller writes a new record to `GoalInSession`. The goal specifies the target type (product specification) and a validation predicate requiring that the thread-size attribute is resolved for the identified product before the session can be marked successful (Table 3).

This goal record is the trace anchor: all subsequent strategy and function records reference `GoalID` either directly (strategy) or transitively (functions via `StrategyID`).

---

<sup>1</sup>For clarity, human-readable function labels (e.g., Analyse With LLM, etc.) are mapped internally to registered callable identifiers (e.g., `func_analyse_data`) defined in the `FunctionTemplateLibrary`.

**Table 3.** Worked example — Goal instance.

| Field           | Value                                                       |
|-----------------|-------------------------------------------------------------|
| GoalID          | 1                                                           |
| SessionID       | 083                                                         |
| GoalName        | Main goal                                                   |
| GoalTarget      | Product specification                                       |
| GoalValidation  | Thread size resolved for identified product                 |
| GoalDescription | Determine the standard thread size of a 4201-16-16 coupling |
| GoalSuccess     | NULL                                                        |

*Strategy selection:*

Control transitions to the **StrategyPlan** node, where the goal instance is matched against **StrategyLibrary**. Because the query contains an explicit high-confidence product identifier, the controller selects *Specification Lookup* rather than broader retrieval strategies. The resulting strategy instance, including an ordered plan (**PlanSteps**) and sufficiency condition, is persisted to **StrategyInSession** (Table 4).

**Table 4.** Worked example — Strategy instance for the query “Vilken gängstorlek har en 4201-16-16?”.

| Field                | Value                                                                                |
|----------------------|--------------------------------------------------------------------------------------|
| StrategyID           | 1                                                                                    |
| GoalID               | 1                                                                                    |
| StrategyName         | Specification Lookup                                                                 |
| StrategyTarget       | Product specification                                                                |
| StrategyDescription  | Direct lookup of a known specification from an identified product code               |
| PlanSteps            | Extract product code; retrieve product details; extract attributes; analyse with LLM |
| SufficiencyCondition | Attribute found and unit normalized                                                  |
| StrategySuccess      | NULL                                                                                 |

This not only provides an explicit, auditable linkage between the abstract strategy template and the executable sequence, but gives a compact, query-specific “recipe” that can be replayed or inspected independently of the underlying graph implementation.

*Function execution and validation:*

Execution proceeds to **FunctionExecute**, which iterates deterministically over the ordered **PlanSteps**. For each step, the invocation is recorded in **FunctionInSession**, while inputs and outputs are materialised as key–value records in **FunctionParametersInSession** and **FunctionOutputInSession**. After each call, **FunctionValidate** enforces basic correctness (e.g., non-empty outputs, expected keys present, count > 0), preventing the plan from advancing when a prerequisite artifact is missing. For readability, Tables 5–8 present compact projections of the corresponding in-session function, parameter, and output records.

i. **Function 1** – Extract Product Number:

The first function processes the query string and extracts the product code (“4201-16-16”) for downstream lookup. The output is stored as a structured

parameter (e.g., keyword output = “4201-16-16”), and the function is marked successful. This isolates the product identifier in a form suitable for database lookup.

(See Table 5.)

- ii. **Function 2** – Query Database: The second function constructs a SQL query against the harvested product table, using “keyword output ” as key. It returns a structured product record, including a specification field (e.g., a JSON or key-value map), and writes both the raw result and a count field to the function output store. Validation at this point requires count > 0; an empty result would immediately fail the strategy and route control to StrategyValidate.

(See Table 6.)

- iii. **Function 3** – Extract Attributes: The third function operates on the retrieved record and projects out the requested attribute. It extracts the *gångstorlek* field (e.g., G1) from the specification structure and stores it as Extracted Attributes. Validation checks that the requested key is present and that its value is non-empty. Failure here would prevent the LLM from synthesizing an answer that satisfies the goal predicate.

(See Table 7.)

- iv. **Function 4** – Analyse With LLM: The final function receives the original query and Extracted Attributes as input. It generates a concise natural-language answer that states the thread size for the identified product, preserving catalogue terminology. The answer text is stored as Analyse Output, and the function is marked successful if the output is non-empty and well-formed.

(See Table 8.)

**Table 5.** Worked example — Extract Product Number.

| Field            | Value                             |
|------------------|-----------------------------------|
| FunctionID       | 1                                 |
| StrategyID       | 1                                 |
| FunctionName     | <code>func_search_products</code> |
| Inputs           | Query string                      |
| Outputs          | Keyword Output = 4201-16-16       |
| ValidationResult | Output non-empty; format valid    |
| FunctionSuccess  | 1                                 |

**Table 6.** Worked example — Query Database.

| Field            | Value                                              |
|------------------|----------------------------------------------------|
| FunctionID       | 2                                                  |
| StrategyID       | 1                                                  |
| FunctionName     | <code>func_get_product_details</code>              |
| Inputs           | Keyword Output = 4201-16-16                        |
| Outputs          | Product record with specification field; count = 1 |
| ValidationResult | count > 0                                          |
| FunctionSuccess  | 1                                                  |

### Function parameter and output schemas.

Beyond runtime traces, the framework maintains declarative schemas for function

**Table 7.** Worked example — Extract Attributes.

| Field            | Value                                           |
|------------------|-------------------------------------------------|
| FunctionID       | 3                                               |
| StrategyID       | 1                                               |
| FunctionName     | <code>func_extract_attributes</code>            |
| Inputs           | Product specification record                    |
| Outputs          | Extracted attributes: <i>gångstorlek = G1</i> " |
| ValidationResult | Attribute present and non-empty                 |
| FunctionSuccess  | 1                                               |

**Table 8.** Worked example — Analyse with LLM.

| Field            | Value                                       |
|------------------|---------------------------------------------|
| FunctionID       | 4                                           |
| StrategyID       | 1                                           |
| FunctionName     | <code>func_analyse_data</code>              |
| Inputs           | Original query and extracted attributes     |
| Outputs          | Natural-language answer stating thread size |
| ValidationResult | Output non-empty; well-formed               |
| FunctionSuccess  | 1                                           |

inputs and outputs. `FunctionParametersLibrary` and `FunctionOutputLibrary` define the expected keys, types, and semantics for each registered function. At execution time, these schemas are used to validate records written to `FunctionParametersInSession` and `FunctionOutputInSession`, enabling early detection of malformed inputs, missing fields, or incompatible outputs before downstream functions are invoked.

Each function invocation results in multiple parameter and output records, materialised as rows in `FunctionParametersInSession` and `FunctionOutputInSession`, respectively.

Below excerpt illustrates how each function invocation materialises concrete output artifacts in `FunctionOutputInSession`, enabling fine-grained inspection, replay, and failure attribution at record level.

**Table 9.** Worked example — `FunctionOutputInSession` (values abridged).

| OID | FID | FunctionName       | OutputName     | OutputValue                                                                  | Type    |
|-----|-----|--------------------|----------------|------------------------------------------------------------------------------|---------|
| 1   | 1   | Extract Product... | keyword output | 4201-16-16                                                                   | string  |
| 2   | 2   | Query Database     | items          | [{product_id: 685, product_code: 4201-16-16, ... }, {product_id: 712, ... }] | string  |
| 3   | 2   | Query Database     | count          | 2                                                                            | integer |
| 4   | 2   | Query Database     | result_source  | products                                                                     | string  |
| 5   | 3   | Extract Attributes | extracted_data | [{product_code: 4201-16-16, specifications: {Gänga: G 1"}}, ... ]            | string  |
| 8   | 4   | Analyse With LLM   | analysis       | Gångstorleken för 4201-16-16 är G 1".                                        | string  |

Each row corresponds to a record in `FunctionOutputInSession`, with complex JSON payloads abridged for readability.

Only representative parameters are shown; auxiliary configuration fields (e.g., joins,

**Table 10.** Worked example — `FunctionParametersInSession` (values abridged).

| PID | FID | FunctionName       | ParamName      | ParamValue                                         | Type    |
|-----|-----|--------------------|----------------|----------------------------------------------------|---------|
| 1   | 1   | Extract Product... | input          | Vilken gängstorlek har en 4201-16-16?              | string  |
| 2   | 2   | Query Database     | query_type     | select                                             | string  |
| 3   | 2   | Query Database     | table          | products                                           | string  |
| 4   | 2   | Query Database     | keyword output | 4201-16-16                                         | string  |
| 8   | 2   | Query Database     | limit          | 100                                                | integer |
| 10  | 3   | Extract Attributes | items          | [[product_id: 685, product_code: 4201-16-16, ...]] | json    |
| 13  | 4   | Analyse With LLM   | task           | advice                                             | string  |
| 16  | 4   | Analyse With LLM   | question       | Vilken gängstorlek har en 4201-16-16?              | string  |

ordering, custom SQL) are omitted for brevity.

**Table 11.** Worked example — Strategy validation.

| Field              | Value                                |
|--------------------|--------------------------------------|
| StrategyID         | 1                                    |
| GoalID             | 1                                    |
| StrategyValidation | All plan steps executed successfully |
| StrategySuccess    | 1                                    |

The combination of `FunctionExecute` and `FunctionValidate` therefore yields a linear, record-level trace: each intermediate artifact (product code, retrieved record, extracted attribute, final response) is persisted and linked to the function invocation that produced it.

This worked example demonstrates how goal satisfaction emerges from validated, sequential agent execution, with all intermediate artifacts persisted in the Relational Control Plane (RCP).

### 3.2. Case Study II:

#### *Query-Driven Engineering Documentation (SAAB)*

This section presents a second industrial case study conducted in collaboration with SAAB, that focuses on query-driven interaction with engineering documentation. Whereas the Hydrosand case addressed structured product catalogues, this case study targets revision-controlled technical documents characterized by heterogeneous and multilingual content, applicability constraints, and multimodal layouts.

Engineering documentation in this domain differs fundamentally from catalog-style product data as in the previous case, requiring reasoning across multiple, heterogeneous documents, with relevant information distributed across textual, tabular, and graphical elements. Answering a single query may therefore require coordinated retrieval, structured extraction, and fusion of evidence from several sources. The case hence motivate explicit orchestration of document-level strategies and function-level operations, with persistent execution traces that capture how intermediate artifacts are produced, combined, and verified before a response is returned.

In compliance with confidentiality requirements, proprietary identifiers, document titles, part numbers, and content values shown in this case study have been anonymized or abstracted. The execution traces, schemas, and control logic presented here never-

theless reflect real runtime artifacts persisted within the RCP.

### 3.2.1. Document Harvesting for Knowledge Representation

This case, also underpinned by a harvested, document-centric knowledge database that preserves *applicability metadata* and *structural context* at extraction time. Source documentation spans heterogeneous formats and layouts, including narrative requirements, dense tables, and figure-referenced descriptions. During ingestion, each document is represented as linked relational units capturing:

- (i) document identity and revision markers,
- (ii) section hierarchy (e.g., headings and paragraphs), and
- (iii) modality-specific artifacts (text passages, table fragments, and figure-referenced elements).

Extracted records retain anchors such as section labels, table captions, and page references, enabling downstream evidence selection to remain location-aware (clause/table/figure context), not merely content-aware.

## Figure Placeholder

**Figure 7.** Knowledge-base schema (Case 2)

This harvested representation supports query-time disambiguation when multiple candidate sources coexist (e.g., overlapping documents or revisions) and provides stable evidence units for subsequent goal–strategy–function execution and validation.

### 3.2.2. Framework instantiation for document querying

The SAAB case reuses the same six-stage goal–strategy–function control loop introduced in Section 2.4. No architectural modifications were required. Instead, the framework is instantiated through a richer set of strategy templates and agent functions tailored to document-centric reasoning.

Compared to the Hydrosand case, the SAAB instantiation exercises deeper strategy plans, parallel execution blocks, and multimodal artifacts. Importantly, no architectural changes were required. The same relational control plane schema governs both cases, demonstrating the generality of the framework.

#### *Strategy templates*

Table 12 summarizes the strategy templates selected from the **StrategyLibrary** for this case. Each template specifies a semantic intent, an executable plan encoded as an ordered (or partially parallel) function sequence, and an explicit sufficiency condition.

#### *Agent function library*

The strategies above are instantiated using callable agent functions registered in the **FunctionTemplateLibrary**. Each function declares a typed input–output contract, enabling deterministic orchestration and schema-level validation.

Table 13 lists the functions relevant to this case study. Human-readable labels are used in the narrative, while callable identifiers are shown verbatim in the tables.

**Table 12.** Strategy templates instantiated for case II domain.

| ID | Strategy                 | StrategyDescription                                                                     | PlanSteps                                                                                                                                                | SufficiencyCondition                                   |
|----|--------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| 1  | Simple Lookup            | Deterministic retrieval of structured information from a single authoritative document. | Extract product code → Find latest document → Table search → Filter table → Assemble table → Analyse with LLM                                            | Requested attribute resolved with supporting evidence. |
| 2  | Enhanced Lookup          | Multi-stage table reasoning with contextual filtering and aggregation.                  | Extract product code → Table search → Filter table → Assemble table → Analyse with LLM                                                                   | Consistent result derived across candidate records.    |
| 3  | Parallel Enhanced Lookup | Latency-aware strategy executing independent steps concurrently.                        | Extract product code → Find latest document → [Table search    Document search] → Filter table → [Analyse with LLM    Convert units] → Aggregate results | Requested attribute resolved and validated.            |
| 4  | Visual Layout Generation | Multimodal retrieval and synthesis of layout-level information.                         | Extract product code → Retrieve image asset → Generate visual layout → Analyse with LLM                                                                  | Layout produced and grounded in source document.       |

**Table 13.** Agent functions used in the SAAB documentation case.

| ID | Function Name                            | Purpose                                                                         | Artifact Produced                     |
|----|------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------|
| 1  | <code>func_extract_product_code</code>   | Identify and validate a product or component identifier from a free-text query. | Validated identifier token.           |
| 2  | <code>func_find_latest_document</code>   | Select the most recent applicable document revision from the corpus.            | Document surrogate ID.                |
| 3  | <code>func_table_search</code>           | Locate candidate tables within a document matching query constraints.           | Table records and metadata.           |
| 4  | <code>func_filter_table</code>           | Apply rule-based and semantic filters to tabular data.                          | Filtered table rows.                  |
| 5  | <code>func_assemble_table</code>         | Merge and normalize table fragments into a unified structure.                   | Structured table artifact (JSON).     |
| 6  | <code>func_generate_visual_layout</code> | Produce a schematic or layout-level representation from document figures.       | Rendered layout artifact.             |
| 7  | <code>func_analyse_data</code>           | Synthesize intermediate artifacts into a grounded natural-language response.    | Final answer text with trace context. |

**Table 14.** Worked example — Goal instance (SAAB case).

| Field           | Value                                                           |
|-----------------|-----------------------------------------------------------------|
| GoalID          | G_01                                                            |
| SessionID       | S_XX                                                            |
| GoalTarget      | Engineering attribute                                           |
| GoalValidation  | Attribute resolved with document provenance                     |
| GoalDescription | Determine attribute X for component <code>COMP_[ID-01]</code> . |
| GoalSuccess     | NULL                                                            |

**Table 15.** Worked example — Strategy instance (SAAB case).

| Field                | Value                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------|
| StrategyID           | STR_01                                                                                                   |
| GoalID               | G_01                                                                                                     |
| StrategyName         | Simple Lookup                                                                                            |
| PlanSteps            | Extract product code; find latest document; table search; filter table; assemble table; analyse with LLM |
| SufficiencyCondition | Attribute resolved with supporting evidence                                                              |
| StrategySuccess      | NULL                                                                                                     |

### 3.2.3. Worked example: end-to-end strategy execution

To illustrate the execution flow concretely, we trace a representative query through the *Simple Lookup* strategy. The query refers to a specific engineering attribute associated with a component identifier; proprietary identifiers are masked.

#### Goal definition

A free-text query is normalized into a structured goal instance at the **Goal-Define** node. The goal specifies the target attribute and a validation predicate requiring that the attribute be resolved with evidence from an authoritative document.

#### Strategy instantiation

The controller selects the *Simple Lookup* strategy based on the presence of a high-confidence component identifier. The resulting strategy instance binds the template plan to the current goal.

**Function execution and validation** Execution proceeds through the ordered function plan. After each invocation, **Function-Validate** enforces schema-level correctness and expected artifact presence before allowing progression.

(Individual function execution tables omitted here for brevity; all steps are logged in the RCP and excerpted below.)

#### Strategy validation

Upon completion of the final analysis step, the controller evaluates the strategy-level sufficiency condition.

#### In-session execution evidence

To demonstrate that all intermediate artifacts are persisted as first-class records, Tables 17 and 18 present abridged excerpts from the **FunctionOutputInSession** and **FunctionParametersInSession** tables.

This case demonstrates that the proposed framework generalizes beyond catalog-style queries to complex, revision-controlled engineering documentation. The same RCP schema supports richer strategies, parallel execution, and multimodal artifacts, while maintaining deterministic orchestration and auditable execution traces under

**Table 16.** Worked example — Strategy validation (SAAB case).

| Field            | Value                                                                    |
|------------------|--------------------------------------------------------------------------|
| StrategyID       | STR_01                                                                   |
| ValidationResult | All plan steps executed successfully and sufficiency condition satisfied |
| StrategySuccess  | 1                                                                        |

**Table 17.** Excerpt from `FunctionOutputInSession` (SAAB case, abridged).

| OID | FID | FunctionName         | OutputName     | OutputValue (masked)   | Type    |
|-----|-----|----------------------|----------------|------------------------|---------|
| 1   | 1   | Extract product code | identifier     | COMP_[ID-01]           | string  |
| 2   | 2   | Find latest document | document_id    | DOC_[D12]              | string  |
| 3   | 3   | Table search         | count          | 2                      | integer |
| 4   | 4   | Filter table         | filtered_rows  | [ROW_01,...]           | json    |
| 5   | 5   | Assemble table       | table_artifact | {fields:...}           | json    |
| 6   | 6   | Analyse with LLM     | analysis       | ‘‘Attribute X is ...’’ | string  |

**Table 18.** Excerpt from `FunctionParametersInSession` (SAAB case, abridged).

| PID | FID | FunctionName         | ParameterName  | ParameterValue (masked) | Type   |
|-----|-----|----------------------|----------------|-------------------------|--------|
| 1   | 1   | Extract product code | input_query    | ‘‘Query_[Q01]’’         | string |
| 2   | 2   | Find latest document | candidate_docs | [DOC_A,...]             | json   |
| 3   | 3   | Table search         | document_id    | DOC_[D12]               | string |
| 4   | 4   | Filter table         | ruleset        | RULE_[R01]              | string |
| 5   | 6   | Analyse with LLM     | assembled_data | {table:...}             | json   |

strict confidentiality constraints.

#### 4. Discussion

The case studies indicate that treating orchestration as persistent data can materially improve the reliability and inspectability of LLM-based extraction pipelines in engineering settings. Across both catalogue-style and revision-controlled documentation, the six-stage control loop forced explicit success criteria (what must be extracted, from which revision scope, and with which evidence granularity) before an answer could be emitted. In practice, this shifted the system from “generate-then-justify” behavior toward “verify-then-summarize” behavior: agent outputs were required to pass schema, provenance, and plausibility checks before being exposed to downstream synthesis.

A key practical implication is that the Relational Control Plane (RCP) reduces engineering effort associated with scaling agentic systems. Because goals, strategies, function invocations, and validation outcomes are stored as relational entities, common operational needs—trace inspection, error attribution, replay, and evaluation—become database queries rather than bespoke logging and instrumentation. This persistence also enables governance-oriented workflows: stakeholders can audit why a particular answer was produced, identify which artifact failed validation, and compare strategy performance over time without modifying orchestration code.

The results also highlight limitations that remain even with validation-centric progression. First, the framework depends on the quality of upstream extraction: table structure reconstruction, OCR errors, and ambiguous typography can still propagate into the harvested database and constrain downstream reasoning. Second, validation predicates require careful design; overly strict predicates may increase latency via retries, while overly permissive predicates may allow subtle unit, revision, or applicability errors to pass. Third, the evaluation presented here focuses on industrially representative cases; broader benchmarking across more document genres (e.g., scanned legacy manuals, engineering drawings, standards with dense cross-references) and a wider set of query types would strengthen generalizability.

Future work will focus on (i) expanding validation to include stronger cross-source consistency checks (e.g., conflict detection and supersession-aware reconciliation across revisions), (ii) improving multimodal grounding for tables and figures through tighter cell/region anchoring, and (iii) developing standardized evaluation suites that score not only answer correctness, but also provenance completeness, revision fidelity, and replay determinism.

In summary, this study presented a domain-agnostic agentic architecture for structured extraction and verification of engineering knowledge. The central contribution is a validation-centric, six-stage control loop coupled with a persistent, SQL-backed Relational Control Plane that stores goals, strategies, agent executions, artifacts, and validation outcomes as first-class relational records. Across two industrial case studies—hydraulic product catalogues and revision-controlled technical documentation—the framework reduced hallucination risk and preserved engineering-critical fidelity (units, revisions, applicability) by enforcing explicit acceptance criteria and retaining fine-grained provenance. By externalizing orchestration as data rather than embedding it solely in code and transient state, the approach improves auditability, deterministic replay, and maintainability, providing a practical foundation for trustworthy generative-AI systems in engineering knowledge management.

## Acknowledgements

We express our gratitude to The Swedish Innovation Agency (Vinnova) for making this research project possible. We also extend our thanks to the industrial partners who provided access to representative technical documentation and domain feedback that informed the evaluation.

We would like to remark that ChatGPT-5 was used in the preparation of this manuscript solely to assist with editing and language refinement. All generated content was carefully reviewed and revised to meet the academic standards of the study.

## Disclosure of interest

The authors report there are no competing interests to declare.

## Author contributions

**Oscar Ikechukwu:** Formal analysis, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

**Mehdi Tarkian:** Conceptualization, Funding acquisition, Methodology, Project administration, Resources, Supervision; Writing – review & editing.

**Sanjay Nambiar:** Conceptualization, Investigation, Writing – review & editing.

**Marie Jonsson:** Funding acquisition, Project administration, Writing – review & editing.

## Funding

This work was supported by Vinnova under Grant 2024-01420 - Data Automation and Retrieval Technology (DART) as part of the research program 'Advanced digitalization - Enabling technologies'.

## ORCID

**Oscar Ikechukwu:**  <https://orcid.org/0009-0000-4905-2344>

**Mehdi Tarkian:**  <https://orcid.org/0009-0003-6305-8621>

**Sanjay Nambiar:**  <https://orcid.org/0000-0003-1745-3869>

**Marie Jonsson:**  <https://orcid.org/0000-0002-6079-2359>

## Data availability statement

The data that support the findings of this study are not publicly available due to confidentiality agreements with the industrial partners. The framework code used in this study is publicly available at [https://github.com/oscik559/Hydroscand\\_Produktbok.git](https://github.com/oscik559/Hydroscand_Produktbok.git). Anonymized artifacts and derived data may be made available by the corresponding author upon reasonable request.

## References

- [1] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv preprint arXiv:200511401. 2020;.
- [2] Huang L, Li Y, Zhang MQ, et al. A survey on hallucination in large language models. arXiv preprint arXiv:231105232. 2023;.
- [3] LiveScience Editorial Team. Explainer: Why ai hallucinates and how researchers are trying to fix it [Livescience (news explainer)]; 2025. URL available via LiveScience website.
- [4] Parthasarathi P, Schick T, Biderman S, et al. Raptor: Recursive abstractive processing for tree-organized retrieval. arXiv preprint arXiv:240118059. 2024; Official code repository available on GitHub.
- [5] Smock B, Pesala R, Abraham R. Pubtables-1m: Towards comprehensive table extraction from unstructured documents. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR); CVF Open Access; 2022. p. 4634–4642.
- [6] Lopez P. Grobid: Machine-learning extraction of pdf documents to structured tei/xml [Documentation and source code available at grobid.readthedocs.io]; 2020. GitHub repository: [github.com/kermitt2/grobid](https://github.com/kermitt2/grobid).
- [7] Galib C, Lopez P, Rödiger D, et al. Automatic identification and normalisation of physical measurements in scientific literature. In: Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM); ACM; 2019. p. 841–850.
- [8] Yao S, Zhao Y, Yu D, et al. React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:221003629. 2022;.
- [9] Schick T, Dwivedi-Yu J, Schärli N, et al. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:230204761. 2023;.
- [10] LangChain/LangGraph Team. Langgraph documentation: Workflows and agent orchestration [Available at [docs.langchain.com/langgraph](https://docs.langchain.com/langgraph)]; 2024.
- [11] LangChain/LangGraph Team. Langgraph persistence: Checkpointers and state storage [Official documentation]; 2024.
- [12] LangChain/LangGraph Contributors. Langgraph: Github repository and feature overview [Github repository: <https://github.com/langchain-ai/langgraph>]; 2024.
- [13] Lim J, Vogel-Heuser B, Kovalenko I. Large language model-enabled multi-agent manufacturing systems. In: 2024 IEEE 20th International Conference on Automation Science and Engineering (CASE); IEEE; 2024. p. 3940–3946.
- [14] Jafari Meimandi K, Aránguiz-Dias G, Kim GR, et al. The measurement imbalance in agentic ai evaluation undermines industry productivity claims. arXiv e-prints. 2025;:arXiv–2506.
- [15] Olujimi PA, Owolawi PA, Mogase RC, et al. Agentic ai frameworks in smmes: A systematic literature review of ecosystemic interconnected agents. AI. 2025;6(6):123.
- [16] Picard C, Edwards KM, Doris AC, et al. From concept to manufacturing: Evaluating vision-language models for engineering design. Artificial Intelligence Review. 2025; 58(9):288.
- [17] Freire SK, Wang C, Foosherian M, et al. Knowledge sharing in manufacturing using large language models: User evaluation and model benchmarking. arXiv preprint arXiv:240105200. 2024;.
- [18] Gkournelos C, Konstantinou C, Makris S. An llm-based approach for enabling seamless human-robot collaboration in assembly. CIRP Annals. 2024;73(1):9–12.
- [19] Singh Gill M, Vyas J, Markaj A, et al. Leveraging llm agents and digital twins for fault handling in process plants. arXiv e-prints. 2025;:arXiv–2505.
- [20] Hydros cand AB. Produktbok 2020 [[https://www.hydros cand.se/se\\_se/fakta-guider/produktboken](https://www.hydros cand.se/se_se/fakta-guider/produktboken)]; 2020. Accessed: 2025-12-03.

## **5. Appendices**

### **Appendix A. Document Extraction**

## Appendix B. Agentic Flow